

AIOS

SEO Automation Platform

System Architecture, Design & Implementation Plan for a cloud-based Audit, Content, and Client Portal system.

MODULE 01

Audit

MODULE 02

Content

MODULE 03

Portal

MODULE 04

Policy Radar

CONTENTS & AT A GLANCE

The system in one page

4

CORE MODULES IN
V1

~90%

CONTENT
AUTOMATION GOAL

3-5

WEEK BUILD
ESTIMATE

24/7

POLICY RADAR
RESEARCH

DOCUMENT CONTENTS

01	Executive Summary	The what & why
02	System Architecture	Full diagram
03	Module - Audit	Reports engine
04	Module - Content	Generate + publish
05	Module - Portal	4 portals
06	Module - Policy Radar	Intelligence
07	Data Model & Reporting	Entities + Sheets
08	Tech Stack	Choices + rationale
09	Infra, Security & Cost	VPS + controls
10	Scope	In / Phase 2 / Out
11	Implementation Roadmap	Week 0-5
12	Risks & Next Steps	Dependencies

THE FOUR MODULES

01 - Audit

20-30+ page SEO audits (on-page, technical, local, AI) run as a cloud job. Technical + actionable in v1.

02 - Content

AI content (AIDA), automated schema, AI images. Publishes to WordPress or a manual PDF. ~90% automated.

03 - Portal

Four role-scoped portals: client, team, manager (optional), admin. Reports, milestones, Fiverr upsells.

04 - Policy Radar

Mandatory core. Auto-researches Google changes, flags them in the knowledge base, and feeds the Command Center. Shared across all deployments.

SECTION 01

Executive Summary

AiOS turns the agency's SEO delivery into a self-running, cloud-hosted platform across four modules - Audit, Content, the Portal, and the mandatory Policy Radar intelligence engine - with Google Sheets reporting and a Command Center where recommendations surface. AIOS is a product deployed per agency client (Danyal, Haseeb, and beyond); the core is shared. The goal is roughly 90% automation, cutting per-page effort from hours to minutes, with a human review checkpoint on the final step.

LOCKED DECISIONS - V1

- **Audit module** reuses the existing, proven audit engine, wrapped as a cloud job / API.
- **Policy Radar** is mandatory core in every deployment - auto-researches Google, flags the knowledge base, feeds the Command Center.
- **Stack:** Next.js portal + FastAPI (Python) + Postgres + Redis, on Docker Compose.
- **API keys** are pre-loaded by the agency into a central vault; no payment gateway in v1.
- **Publishing:** WordPress REST API in v1, plus a manual PDF path. **Images** AI-generated with alt text.
- **Onboarding:** agency-provisioned accounts; no public self-signup.
- **Financial audit report** is documented now, built in Phase 2.
- **Off-Page module** is out of scope for v1; documented for later.
- **Free vs paid** audit tiers are set by the agency and gate the costly integrations.
- **Deliverable** is this internal founder-grade architecture + build plan.

The problem

Manual audits and content take hours per page and every automated run burns API budget that is hard to predict.

The system

One cloud platform: click to audit, generate + publish content, and give every client a live portal - all on cost-controlled infrastructure.

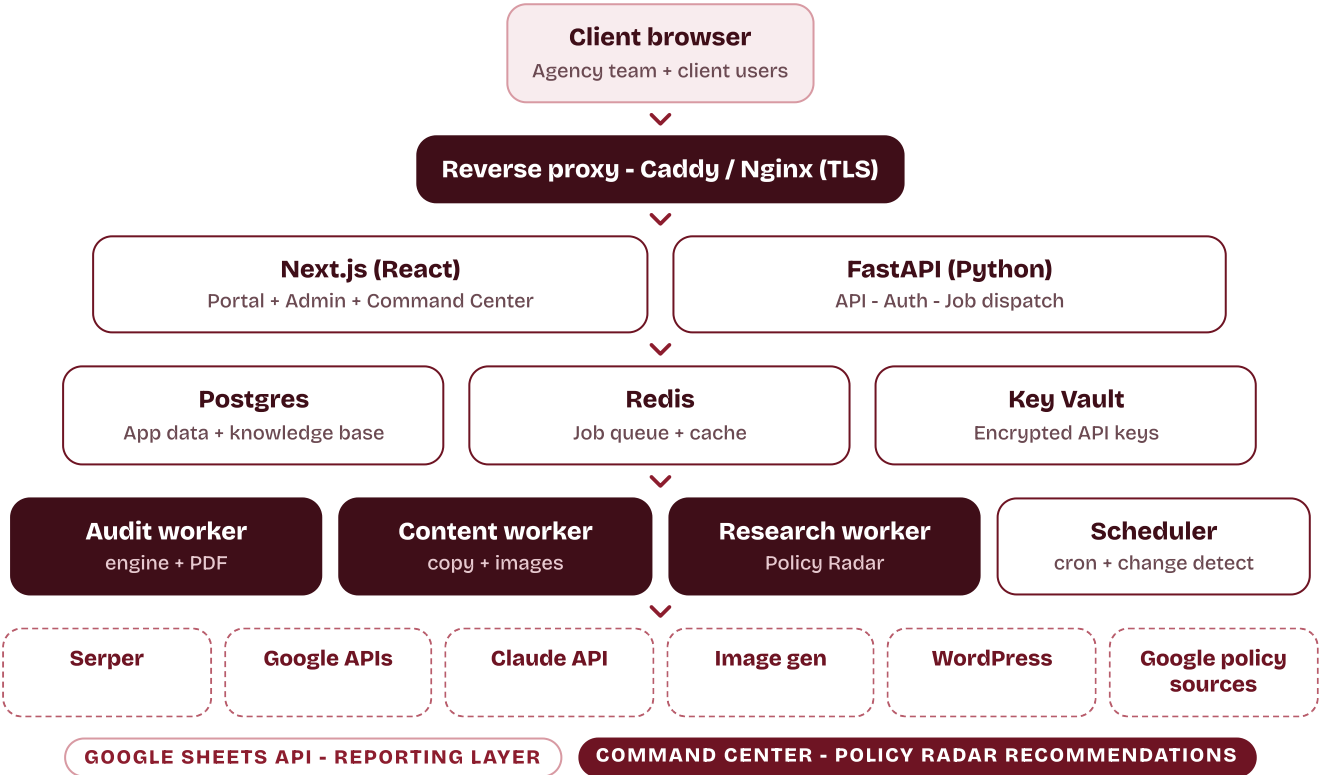
The payoff

Minutes per page, a scalable client base, and a branded self-serve experience that funnels upsells to the agency's Fiverr gigs.

SECTION 02

System Architecture

A browser talks to the Next.js app. Next.js calls FastAPI for all logic and auth. FastAPI uses Postgres and enqueues slow work into Redis. Python workers run the Audit, Content, and Policy Radar research engines against external APIs through a shared, encrypted key vault. A scheduler drives Policy Radar. Everything runs as Docker Compose services on one VPS behind a TLS proxy.

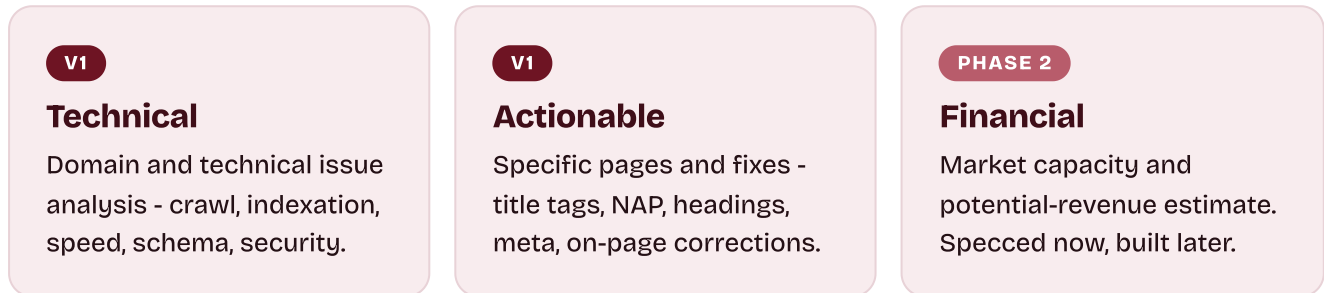


SECTION 03 - MODULE 01

Audit

The existing, proven audit engine - crawler, analyzers, scorers, and the house-styled PDF generator - wrapped as a cloud job. Clients click **Run audit** in the portal; a worker executes it with the agency's keys and returns a 20-30+ page report as a web page and a downloadable PDF.

REPORT TYPES



HOW A RUN FLOWS



Then the platform reacts automatically

The report appears in the portal, the client's milestone advances, the Google Sheet updates, and the client is notified - no manual steps. Free vs paid tier decides which paid integrations (Serper, PageSpeed, Places, NL) are allowed to run, keeping spend controlled.

Coverage

On-page, technical, local, and AI / GEO elements. Off-page is deferred to a future phase.

Reuse, not rebuild

The engine already exists and is tested. v1 adds a job wrapper + API around it - the fastest safe path and the reason the timeline holds.

SECTION 04 - MODULE 02

Content

A cloud AI content engine on the same cost-controlled queue. It drafts with proven copy frameworks, builds schema, generates images, then either exports a manual PDF or publishes straight to WordPress.

GENERATION PIPELINE



TWO PUBLISHING PATHS



Human review checkpoint

Every content job pauses for a quick human review before publish. That checkpoint is the deliberate 10% - it catches hallucination and off-brand copy while keeping the throughput of full automation.

SECTION 05 - MODULE 03

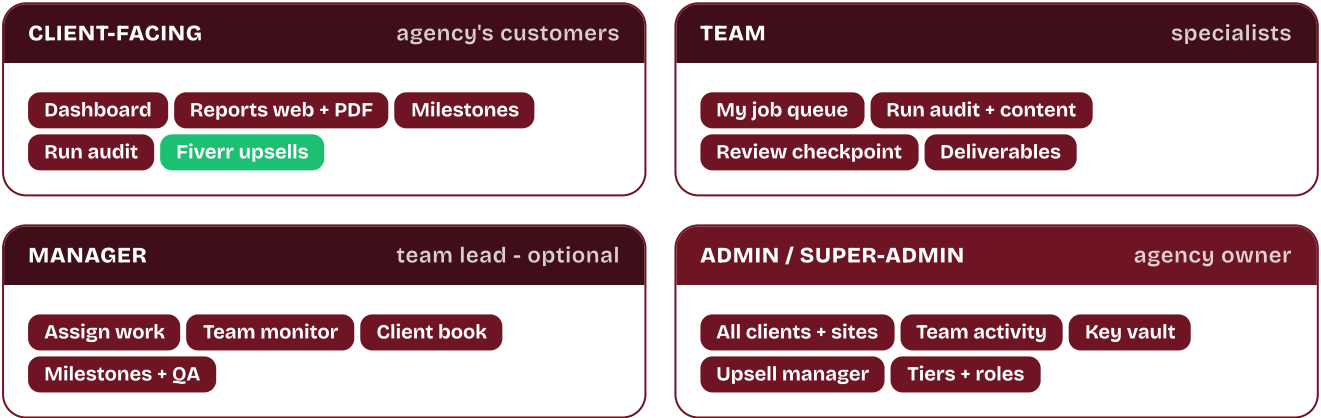
Portal

One application, **up to four role-scoped portals** on a shared login and data layer. Access widens from client to super-admin. Accounts are agency-provisioned - no public signup. The Manager portal is optional in v1; if deferred, its scope folds into Admin.

ACCESS SCOPES - ONE APP, FOUR PORTALS



THE FOUR PORTALS



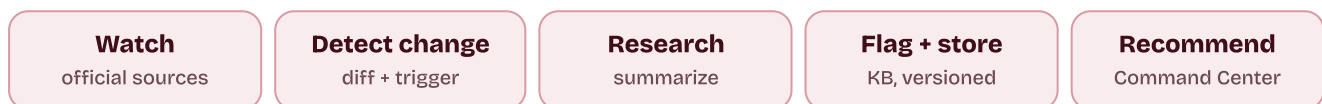
Upsells route to Fiverr - by design

Client upsell buttons link to the agency's Fiverr gigs, not internal services - a deliberate scope-call decision that preserves the agency's Fiverr-centered public brand.

SECTION 06 - MODULE 04 - MANDATORY CORE

Policy Radar (Intelligence)

An always-on engine that keeps every AIOS deployment current with Google. It is platform core, not a per-client feature: it ships in every deployment (Danyal, Haseeb, and beyond) and feeds recommendations into the **Command Center**. When Google changes or releases something, it auto-researches that change and recommends what to do.

THE LOOP**NATIONAL FLAGGING - EVERY ENTRY TAGGED**

- **Severity** - critical / major / minor / info.
- **Category** - algorithm / policy / technical / content / local / GEO.
- **Region / nation scope** - global vs specific countries, so advice filters to each client's market.
- **Source cited** - Search Status Dashboard, Search Central, QRG. Nothing applied on an unverifiable claim.

COMMAND CENTER**Every recommendation, in one place**

Shows all current best practices as a baseline, plus new or changed recommendations triggered by Google moves - what changed, why it matters, regions affected, and the action.

Closed loop

A recommendation can add or update audit checks, adjust content guidance, or raise a client advisory. Human-confirmed before it changes live checks.

SECTION 07

Data Model & Reporting

Multi-tenant under a single agency. A compact relational core drives both faces of the portal; Google Sheets is the cross-cutting reporting layer.

CORE ENTITIES

users

role: super_admin / manager / team_member / client. Auth + access.

clients

The agency's customers. A client owns one or more sites.

sites

domain, cms_type, encrypted WordPress credentials.

audits

type, tier, status, run_uuid, artifact_path, scores.

content_jobs

content_type, framework, target, status, published_url, cost.

milestones

title, status, auto_source, updated_at - drives the client timeline.

upsells

title, description, fiverr_url, active flag.

api_keys

agency-level encrypted vault for every provider.

activity_log

actor, action, target, timestamp - feeds the admin monitor.

POLICY RADAR ENTITIES (INTELLIGENCE)

kb_entries

source, url, category, **severity**, **region_flags[]**, summary, version, hash.

recommendations

kb_entry_id, scope, target_module (audit/content), action, status.

policy_sources

name, url, type, last_checked, last_hash - drives change detection.

change_events

source_id, detected_at, diff_ref, triggered_job - Google-move trail.

REPORTING - GOOGLE SHEETS

Audit scores



Google Sheets API
Service account



Shareable dataviz
Per client / master

Content-job status and milestone state push to the same sheets, giving the agency and clients familiar, shareable visualizations without building bespoke charts in v1.

SECTION 08

Tech Stack

One Python spine for all logic, AI, and every engine; Next.js for a fast modern UI; a queue to isolate expensive work. Chosen to run cleanly on a single VPS with predictable spend.

LAYER	CHOICE	WHY IT WAS CHOSEN
Portal + Admin UI	Next.js (React)	Modern, fast, server-rendered; one UI serves both the client and admin faces.
API + Auth	FastAPI (Python)	Shares a language with both engines; typed and fast; simple job dispatch.
Audit engine	Existing engine (Python)	Proven and tested; wrapped as a worker job rather than rebuilt.
Content engine	Python + Claude + image model	Same spine; AIDA copy, automated schema, AI images with alt text.
Database	Postgres	Reliable relational store for multi-tenant data.
Queue + cache	Redis	Job queue, concurrency caps, and caching of costly API calls.
Publishing	WordPress REST API	The v1 CMS target; app-password auth per client site.
Intelligence	Scheduler + web research + Claude	Policy Radar: watch Google, research changes, flag the KB, recommend.
Reporting	Google Sheets API	Familiar, shareable client-facing dataviz with no custom charting.
Deploy	Docker Compose + Caddy TLS	Single-box, cost-controlled, simple to operate and hand off.

One spine

API, workers, and all engines are Python - shared libraries, one mental model, easy hiring.

Queue-isolated

Slow, costly API work runs off the request path so the UI stays fast and spend stays capped.

Single-box simple

Everything is a Compose service on the VPS - trivial to deploy, back up, and reason about.

SECTION 09

Infra, Security & Cost Control

Cost control is the reason the platform is cloud-based. The queue, the shared vault, and caching exist to make automated API spend predictable.

DEPLOYMENT

Docker Compose services: web (Next.js), api (FastAPI), worker (audit + content + research), scheduler (Policy Radar cron + change detect), postgres, redis, proxy (Caddy/Nginx, auto-TLS). Artifacts on the VPS volume; add object storage later if needed.

Recommended VPS

Ubuntu 22.04+, 4 vCPU / 8-16 GB RAM / 100-160 GB NVMe SSD. Hetzner / DigitalOcean / Contabo class. Adan provisions once access is granted.

COST CONTROL - THE REASON FOR CLOUD

- Central, agency-owned key vault - no per-client keys.
- Queue concurrency limits + per-run budget caps.
- Redis caching of expensive API responses.
- Free / paid tiers gate the costly integrations.
- Per-job cost estimation + logging, shown to admin.

SECURITY**Secrets**

Encrypted key vault + WordPress creds. Keys never logged. Role-based access across the app.

Resilience

Nightly Postgres backups, container restart policies, documented restore, TLS everywhere.

Trust boundary

Audited-site content and generated copy are treated as data, not instructions.

SECTION 10

Project Scope

A tight v1 protects the 3-5 week timeline. Everything else is parked in a named backlog so new requests never quietly derail the core build.

IN SCOPE - V1	PHASE 2	OUT OF SCOPE
■ Audit module (technical + actionable) as a cloud job / API. ■ Content: AIDA drafting, automated schema, AI images. ■ Manual-PDF + WordPress publishing, with review checkpoint. ■ Portal: client, team, admin (manager optional). ■ Policy Radar + Command Center (mandatory core). ■ Agency-provisioned accounts, Google Sheets reporting. ■ Central API key vault + cost controls.	■ Financial audit report (market capacity + revenue). ■ Off-Page module. ■ Fiverr client-data import. ■ Additional CMS connectors (Shopify / Webflow / headless). Documented now so the next call can turn them into a Phase 2 plan.	■ Self-serve payment gateway (Stripe). ■ Public self-signup. Scope discipline New feature requests are logged to the Phase 2 backlog, not slipped into v1 - the guardrail that keeps the timeline honest.

SECTION 11

Implementation Roadmap

Six tracks over 3-5 weeks. Each closes on a concrete, demoable exit criterion - so progress is provable, not assumed.

WEEK
0

Pre-reqs - Danyal-dependent

VPS access, API keys loaded, sample Fiverr client data, a test WordPress site, subdomain + repo/CI.

Exit: environment reachable and provisioned.

WEEK
1

Foundations

Docker Compose, Postgres schema, auth + roles, Next.js shell, FastAPI skeleton, audit engine wrapped as a worker job.

Exit: an audit triggers via an internal endpoint and returns a PDF.

WEEK
2

Audit in the portal

Client dashboard, reports list + web/PDF view, run-audit flow (free/paid), admin client management, API key vault.

Exit: a provisioned client logs in, runs an audit, downloads the PDF.

WEEK
3

Content module

Content engine (AIDA + schema + AI images), manual-PDF path, WordPress connector, content-job UI + review checkpoint.

Exit: generate a service page, review, publish to a test WordPress site.

WEEK
4

Portal completion + Policy Radar

Milestones, Fiverr upsells, Google Sheets reporting, team-activity monitor, cost tracking, and the Intelligence engine: scheduler + change detection, flagged KB, Command Center.

Exit: full client journey + admin oversight + Command Center recommendations live.

WEEK
5

Hardening + handoff

Security pass, backups, performance, UAT with Danyal, docs / runbook, production deploy, buffer.

Exit: production live, Danyal onboarded.

SECTION 12

Risks & Next Steps

RISKS & MITIGATIONS

RISK	MITIGATION
API cost overruns	Budget caps, caching, tier gating, per-job cost logging.
WordPress site variability	App-password auth, per-site config, graceful fallback to the manual PDF.
Content hallucination	Human review checkpoint, schema validation, evidence-based audit rules.
Policy Radar false positives	Every entry cites its source; recommendations human-confirmed in the Command Center before changing live checks.
Scope creep vs timeline	New requests parked to the named Phase 2 backlog.
Single VPS point of failure	Backups, restart policies, documented restore; scale out later.
Fiverr has no public API	Manual / CSV client-data import in v1.

IMMEDIATE NEXT STEPS

Danyal

VPS access; load API keys (Serper, Google, Anthropic, image gen, Sheets); share Fiverr client data; provide a test WordPress site + app password; document the Off-Page module.

Adan

Provision + harden the VPS; set up Docker + CI/CD; begin Week 1 foundations.

Zain

Next call in 1-2 weeks to review the Off-Page module documentation and shape Phase 2.